

Machine Learning for Mechanical Engineers at CoEP Session 7: Sklearn Workflow

Session 7: Sklearn Workflow

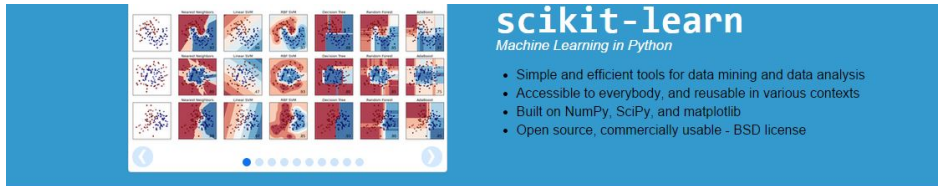
Introduction to Scikit-Learn aka Sklearn

scikit.learn

- Open source machine learning library for Python.
- Features various classification, regression and clustering algorithms



Machine Learning in Python



Classification

Identifying to which category an object belongs to.

Applications: Spam detection, Image recognition.
Algorithms: SVM, nearest neighbors, random forest, ... — Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.
Algorithms: SVR, ridge regression, Lasso, ... — Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes
Algorithms: k-Means, spectral clustering, mean-shift, ... — Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency
Algorithms: PCA, feature selection, non-negative matrix factorization. — Examples

Model selection

Comparing, validating and choosing parameters and models.

Goal: Improved accuracy via parameter tuning
Modules: grid search, cross validation, metrics. — Examples

Preprocessing

Feature extraction and normalization.

Application: Transforming input data such as text for use with machine learning algorithms.
Modules: preprocessing, feature extraction — Examples

Scikit-Learn's Estimator API

The steps in using the Scikit-Learn estimator API are as follows

- Choose a class of model by importing the appropriate estimator class from Scikit-Learn.
- Choose model hyper-parameters by instantiating this class.
- Arrange data into a features matrix and target vector.
- Fit the model to your data by calling the fit() method of the model instance.
 - For supervised learning, predict labels for unknown data using the predict().
 - For unsupervised learning, transform or infer properties of the data using transform() or predict().
- The Estimator.fit method sets the state of the estimator based on the training data.
- Usually, the data is comprised of a two-dimensional numpy array X of shape (n_samples, n_predictors) that holds the so-called feature matrix and a one-dimensional numpy array y that holds the responses.
- Some estimators allow the user to control the fitting behavior.

Estimator

- fit(X,y) sets the state of the estimator.
- X is usually a 2D numpy array of shape (num_samples, num_features).
- y is a 1D array with shape (n_samples,)
- predict(X) returns the class or value

```
class Estimator(object):
    def fit(self, X, y=None):
        """Fits estimator to data. """
        # set state of 'self'
        return self
    def predict(self, X):
        """Predict response of 'X'. """
        # compute predictions 'pred'
        return pred
```

Sklearn: Installations

Scikit Learn: Installation and setup

Install Anaconda distribution having Python 3.5 \$ conda install numpy scipy matplotlib scikit-learn ipython-notebook

Sklearn Paradigm

Scikit-learn provides an object-oriented interface centered around the concept of an Estimator.

Checking your installation

```
import numpy
print('numpy:', numpy.__version__)

import scipy
print('scipy:', scipy.__version__)

import matplotlib
print('matplotlib:', matplotlib.__version__)

import sklearn
print('scikit-learn:', sklearn.__version__)

import seaborn
print('seaborn', seaborn.__version__)
```

Data Preprocessing with Scikit-Learn

Data Preprocessing for Pima-Indians Diabetes Dataset

(Ref: How To Prepare Your Data For Machine Learning in Python with Scikit-Learn - Jason Brownlee)

Read Data

Import Pima Indians dataset and split into Features (X) and target (Y)

```
import pandas
import scipy
import numpy

url =
    "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']

dataframe = pandas.read_csv(url, names=names)
array = dataframe.values

# separate array into input and output components
X = array[:,0:8]
Y = array[:,8]
```

Rescale Data

- Referred to as normalization and attributes are often rescaled into the range between 0 and 1.
- This is useful for optimization algorithms in used in the core of machine learning algorithms like gradient descent.
- It is also useful for algorithms that weight inputs like regression and neural networks and algorithms that use distance measures like K-Nearest Neighbors.

Rescale Data

Rescale your data using scikit-learn using the MinMaxScaler class.

```
# Rescale data (between 0 and 1)
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1))
rescaledX = scaler.fit_transform(X)

# summarize transformed data
numpy.set_printoptions(precision=3)
print(rescaledX[0:5,:])
```

Rescale Data

After rescaling you can see that all of the values are in the range between 0 and 1.

```
[[ 0.353  0.744  0.59   0.354  0.    0.501  0.234  0.483]
 [ 0.059  0.427  0.541  0.293  0.    0.396  0.117  0.167]
 [ 0.471  0.92   0.525  0.    0.    0.347  0.254  0.183]
 [ 0.059  0.447  0.541  0.232  0.111  0.419  0.038  0.   ]
 [ 0.    0.688  0.328  0.354  0.199  0.642  0.944  0.2   ]]
```

Standardize Data

- Standardization is a useful technique to transform attributes with a Gaussian distribution and differing means and standard deviations to a standard Gaussian distribution with a mean of 0 and a standard deviation of 1.
- It is most suitable for techniques that assume a Gaussian distribution in the input variables and work better with rescaled data, such as linear regression, logistic regression and linear discriminate analysis.

Intuition

Rescale (MinMaxScaler) and Standardize (StandardScaler) are easy to mix up: rescaling squeezes every value into a fixed [0,1] box regardless of shape, while standardizing centers the data on 0 and measures everything in “number of standard deviations”: which is why standardized values can still go negative or exceed 1, unlike rescaled ones.

Standardize Data

Standardize data using scikit-learn with the StandardScaler class.

```
# Standardize data (0 mean, 1 stdev)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X)
rescaledX = scaler.transform(X)

# summarize transformed data
numpy.set_printoptions(precision=3)
print(rescaledX[0:5,:])
```

Standardize Data

The values for each attribute now have a mean value of 0 and a standard deviation of 1.

```
[[ 0.64   0.848  0.15   0.907 -0.693  0.204  0.468  1.426]
 [-0.845 -1.123 -0.161  0.531 -0.693 -0.684 -0.365 -0.191]
 [ 1.234  1.944 -0.264 -1.288 -0.693 -1.103  0.604 -0.106]
 [-0.845 -0.998 -0.161  0.155  0.123 -0.494 -0.921 -1.042]
 [-1.142  0.504 -1.505  0.907  0.766  1.41   5.485 -0.02 ]]
```

Normalize Data

- Normalizing in scikit-learn refers to rescaling each observation (row) to have a length of 1 (called a unit norm in linear algebra).
- This preprocessing can be useful for sparse datasets (lots of zeros) with attributes of varying scales when using algorithms that weight input values such as neural networks and algorithms that use distance measures such as K-Nearest Neighbors.

Normalize Data

Normalize data in Python with scikit-learn using the Normalizer class.

```
# Normalize data (length of 1)
from sklearn.preprocessing import Normalizer

scaler = Normalizer().fit(X)
normalizedX = scaler.transform(X)
# summarize transformed data
numpy.set_printoptions(precision=3)
print(normalizedX[0:5,:])
```

Normalize Data

Each row is rescaled so that the sum of the squares of its values equals 1: the row's direction is preserved, only its length changes. A simple worked example, for one row of two features. Applied to every row of X, this is exactly what **normalizedX** above contains: each row now has unit length, so no single value in it can exceed 1 in magnitude.

```
row = [4, 3]                # length = sqrt(4^2 + 3^2) = 5
normalized_row = [4/5, 3/5]  # = [0.8, 0.6]
# check: 0.8^2 + 0.6^2 = 1.0
```

Binarize Data (Make Binary)

- You can transform your data using a binary threshold. All values above the threshold are marked 1 and all equal to or below are marked as 0.
- This is called binarizing your data or threshold your data. It can be useful when you have probabilities that you want to make crisp values.
- It is also useful when feature engineering and you want to add new features that indicate something meaningful.

Binarize Data

Create new binary attributes in Python using scikit-learn with the Binarizer class.

```
# binarization
from sklearn.preprocessing import Binarizer

binarizer = Binarizer(threshold=0.0).fit(X)
binaryX = binarizer.transform(X)
# summarize transformed data
numpy.set_printoptions(precision=3)
print(binaryX[0:5,:])
```

Binarize Data

You can see that all values equal or less than 0 are marked 0 and all of those above 0 are marked 1.

```
[[ 1.  1.  1.  1.  0.  1.  1.  1.]
 [ 1.  1.  1.  1.  0.  1.  1.  1.]
 [ 1.  1.  1.  0.  0.  1.  1.  1.]
 [ 1.  1.  1.  1.  1.  1.  1.  1.]
 [ 0.  1.  1.  1.  1.  1.  1.  1.]
```

Quick Check: Data Preprocessing

Think About It

You fit a **StandardScaler** on your training data, then need to preprocess the test set before evaluating your model. Should you call **scaler.fit(X_test)** again, or reuse the scaler fitted on the training data?

Answer

Always reuse the scaler fitted on the training data (**scaler.transform(X_test)**, no re-fit). Fitting a new scaler on the test set would compute a *different* mean/std from data the model has never seen, so the same raw value could be scaled differently between train and test – silently corrupting the features the model was trained on. Fitting only ever happens once, on training data; every other split reuses that same fitted scaler.

Model Evaluation with Scikit-Learn

Classification Model Evaluation for Pima-Indians Diabetis Dataset

(Ref: Metrics To Evaluate Machine Learning Algorithms in Python - Jason Brownlee)

Read Data

Import Pima Indians dataset and split into Features (X) and target (Y)

```
import pandas
import scipy
import numpy

url =
    "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']

dataframe = pandas.read_csv(url, names=names)
array = dataframe.values

# separate array into input and output components
X = array[:,0:8]
Y = array[:,8]
```

Classification Accuracy

- Classification accuracy is the number of correct predictions made as a ratio of all predictions made.
- This is the most common evaluation metric for classification problems, it is also the most misused.
- It is really only suitable when there are an equal number of observations in each class (which is rarely the case) and that all predictions and prediction errors are equally important, which is often not the case.

Classification Accuracy

Accuracy score of approximately 77% accurate

```
from sklearn import model_selection
from sklearn.linear_model import LogisticRegression

seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True, random_state=seed)
model = LogisticRegression()
scoring = 'accuracy'
results = model_selection.cross_val_score(model, X, Y, cv=kfold,
                                          scoring=scoring)
print("Accuracy: %.3f (%.3f)" % (results.mean(), results.std()))

Accuracy: 0.770 (0.048)
```

Logarithmic Loss

- Logarithmic loss (or logloss) is a performance metric for evaluating the predictions of probabilities of membership to a given class.
- The scalar probability between 0 and 1 can be seen as a measure of confidence for a prediction by an algorithm. Predictions that are correct or incorrect are rewarded or punished proportionally to the confidence of the prediction.

Logarithmic Loss

Smaller logloss is better with 0 representing a perfect logloss. As mentioned above, the measure is inverted to be ascending when using the `cross_val_score()` function.

```
seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True, random_state=seed)
model = LogisticRegression()
scoring = 'neg_log_loss'
results = model_selection.cross_val_score(model, X, Y, cv=kfold,
                                          scoring=scoring)
print("Logloss: %.3f (%.3f)" % (results.mean(), results.std()))

Logloss: -0.493 (0.047)
```

Area Under ROC Curve

- Area under ROC Curve (or AUC for short) is a performance metric for binary classification problems.
- The AUC represents a model's ability to discriminate between positive and negative classes. An area of 1.0 represents a model that made all predictions perfectly. An area of 0.5 represents a model as good as random.
- ROC can be broken down into sensitivity and specificity. A binary classification problem is really a trade-off between sensitivity and specificity.
 - Sensitivity is the true positive rate also called the recall. It is the number instances from the positive (first) class that actually predicted correctly.
 - Specificity is also called the true negative rate. Is the number of instances from the negative class (second) class that were actually predicted correctly.

Intuition

AUC = 0.5 isn't "50% accurate": it means the model's ranking of positive vs. negative cases is no better than a coin flip. AUC is threshold-free: it asks "if I pick one random positive and one random negative example, how often does the model score the positive one higher?": which is why it stays meaningful even when the two classes are wildly imbalanced, unlike plain accuracy.

Area Under ROC Curve

The AUC is relatively close to 1 and greater than 0.5, suggesting some skill in the predictions.

```
seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True, random_state=seed)
model = LogisticRegression()
scoring = 'roc_auc'
results = model_selection.cross_val_score(model, X, Y, cv=kfold,
                                          scoring=scoring)
print("AUC: %.3f (%.3f)" % (results.mean(), results.std()))

AUC: 0.824 (0.041)
```

Confusion Matrix

- The confusion matrix is a handy presentation of the accuracy of a model with two or more classes.
- For example, a machine learning algorithm can predict 0 or 1 and each prediction may actually have been a 0 or 1.
- Predictions for 0 that were actually 0 appear in the cell for prediction=0 and actual=0, whereas predictions for 0 that were actually 1 appear in the cell for prediction = 0 and actual=1. And so on.

Confusion Matrix

The majority of the predictions fall on the diagonal line of the matrix (which are correct predictions).

```
from sklearn.metrics import confusion_matrix

test_size = 0.33
seed = 7
X_train, X_test, Y_train, Y_test = model_selection.train_test_split(X, Y,
    test_size=test_size, random_state=seed)
model = LogisticRegression()
model.fit(X_train, Y_train)
predicted = model.predict(X_test)
matrix = confusion_matrix(Y_test, predicted)
print(matrix)

[[141  21]
 [ 41  51]]
```

Classification Report

- Scikit-learn does provide a convenience report when working on classification problems to give you a quick idea of the accuracy of a model using a number of measures.
- The classification_report() function displays the precision, recall, f1-score and support for each class.

Classification Report

You can see good prediction and recall for the algorithm.

```
from sklearn.metrics import classification_report

test_size = 0.33
seed = 7
X_train, X_test, Y_train, Y_test = model_selection.train_test_split(X, Y,
    test_size=test_size, random_state=seed)
model = LogisticRegression()
model.fit(X_train, Y_train)
predicted = model.predict(X_test)
report = classification_report(Y_test, predicted)
print(report)

              precision    recall  f1-score   support


```

	0.0	0.77	0.87	0.82	162
	1.0	0.71	0.55	0.62	92
avg / total		0.75	0.76	0.75	254

Regression Model Evaluation for Boston House Price Dataset

(Ref: Metrics To Evaluate Machine Learning Algorithms in Python - Jason Brownlee)

Read Data

Import Boston House Price dataset and split into Features (X) and target (Y)

```
import pandas

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.data"
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD',
    'TAX', 'PTRATIO', 'B', 'LSTAT', 'MEDV']
dataframe = pandas.read_csv(url, delim_whitespace=True, names=names)
array = dataframe.values
X = array[:,0:13]
Y = array[:,13]
```

Mean Absolute Error

- The Mean Absolute Error (or MAE) is the sum of the absolute differences between predictions and actual values. It gives an idea of how wrong the predictions were.
- The measure gives an idea of the magnitude of the error, but no idea of the direction (e.g. over or under predicting).

Mean Absolute Error

A value of 0 indicates no error or perfect predictions. Like logloss, this metric is inverted by the cross_val_score() function.

```
from sklearn.linear_model import LinearRegression

seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True, random_state=seed)
model = LinearRegression()
scoring = 'neg_mean_absolute_error'
results = model_selection.cross_val_score(model, X, Y, cv=kfold,
    scoring=scoring)
print("MAE: %.3f (%.3f)" % (results.mean(), results.std()))

MAE: -4.005 (2.084)
```

Mean Squared Error

- The Mean Squared Error (or MSE) is much like the mean absolute error in that it provides a gross idea of the magnitude of error.
- Taking the square root of the mean squared error converts the units back to the original units of the output variable and can be meaningful for description and presentation. This is called the Root Mean Squared Error (or RMSE).

Mean Squared Error

This metric too is inverted so that the results are increasing. Remember to take the absolute value before taking the square root if you are interested in calculating the RMSE.

```
seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True, random_state=seed)
model = LinearRegression()
scoring = 'neg_mean_squared_error'
results = model_selection.cross_val_score(model, X, Y, cv=kfold,
                                          scoring=scoring)
print("MSE: %.3f (%.3f)" % (results.mean(), results.std()))

MSE: -34.705 (45.574)
```

R^2 Metric

- The R^2 (or R Squared) metric provides an indication of the goodness of fit of a set of predictions to the actual values. In statistical literature, this measure is called the coefficient of determination.
- $R^2 = 1$ is a perfect fit and $R^2 = 0$ means the model does no better than always predicting the mean; but R^2 is *not* bounded below by 0: a model that fits worse than that mean-only baseline (common on a bad fold) can score negative, which is exactly why the example below has such a large standard deviation relative to its mean.

R^2 Metric

Unlike MAE and MSE above, 'r2' is not negated by `cross_val_score()`: higher is already better, so this mean can be read directly. The large standard deviation relative to the mean here is exactly the negative- R^2 -on-a-bad-fold effect described on the previous slide.

```
seed = 7
kfold = model_selection.KFold(n_splits=10, shuffle=True, random_state=seed)
model = LinearRegression()
scoring = 'r2'
results = model_selection.cross_val_score(model, X, Y, cv=kfold,
                                          scoring=scoring)
print("R^2: %.3f (%.3f)" % (results.mean(), results.std()))

R^2: 0.203 (0.595)
```

Quick Check: Model Evaluation

Think About It

The Pima diabetes dataset has roughly twice as many non-diabetic (class 0) cases as diabetic (class 1) cases. A model that always predicts class 0, no matter the input, could reach around 65% classification accuracy. Which of the metrics on the earlier slides would immediately expose this model as useless, and why does plain accuracy fail to?

Answer

AUC would immediately expose it: a constant-output model can't rank any example above another, so it scores $AUC \approx 0.5$, no better than random, regardless of class imbalance. The confusion matrix would show the same story directly: an entire empty row or column, since class 1 is never predicted. Accuracy fails here because it only counts how many predictions matched, and on an imbalanced dataset "always predict the majority class" matches a lot of the time by construction: exactly why accuracy is called out on the first slide of this section as "the most misused" metric.